Order-Flow Alpha and a Tick-Level Execution Backtester

A step-by-step tutorial and results report

Binance USD-M perpetual XRPUSDT · 2024-02-01 to 2024-02-10 · 30.5 million quote updates and 1.12 million trades

0.29	−10.6 bp	−32%	148M/s
test-day rank IC of the 5-second forecast	taker strategy, net per round trip	slippage vs passive execution (30 s window)	events replayed per second, one core

Project folder: `orderflow-alpha-backtester` · Python, NumPy, pandas, Numba, Matplotlib · report generated from the project's own result files

Contents

1. Executive summary 3

2. Concepts you need first 3

2.1 The limit order book and the touch . . . 3

2.2 Three order-flow signals . . . 4

2.3 Makers, takers, fees and latency . . . 4

2.4 Adverse selection and queue position . . . 4

2.5 Measuring prediction . . . 4

2.6 Measuring execution cost . . . 4

3. Project layout and setup 5

4. Step 1: Getting the data 6

5. Step 2: Point-in-time features 7

6. Step 3: Signal research 8

6.1 Which features predict, and how far ahead . . . 8

6.2 How big the predicted moves are . . . 9

6.3 A combined linear forecast . . . 9

7. Step 4: The matching engine 10

7.1 Event loop . . . 10

7.2 Fill rules . . . 10

7.3 Unit tests . . . 11

8. Step 5: Standalone strategies 12

9. Step 6: The execution overlay 13

9.1 Design . . . 13

9.2 Results . . . 14

10. Limitations and next steps 16

10.1 Limitations . . . 16

10.2 Natural extensions . . . 17

Appendix A. Console output of the sweep 17

Appendix B. Glossary 17

1. Executive summary

This project asks a practical question that every short-horizon quant faces: **a signal predicts the next few seconds of price movement, but does it survive real trading?** To answer it, the project downloads raw tick data, builds order-flow features, measures their predictive power out of sample, and then replays every quote and trade through a purpose-built matching engine with latency, queue position and exchange fees.

The answer has three parts:

1. **The signal is real.** Level-1 queue imbalance has a Spearman IC of 0.273 with the 5-second mid move on training days, 0.283 on validation days and 0.284 on test days. A linear model of nine features reaches a test IC of 0.289 and an out-of-sample R^2 of 5.5%.
2. **It cannot be traded on its own.** The forecast rarely exceeds half a tick, while the spread is one tick (about 1.96 bp) and fees add 4 to 10 bp per round trip. The best taker configuration loses 10.6 bp per trade; the best maker configuration loses 6.9 bp.
3. **It is valuable for execution.** Across 25,875 simulated child orders, using the forecast to decide when to wait and when to cross cuts slippage by 32% against plain passive execution and total cost by 1.42 bp (24%) against market orders, in a 30-second window. A random-sign placebo makes costs worse, so the gain comes from the signal.

Area	Headline result
Signal	Queue imbalance IC 0.284 at 5 s on test days; stable across all splits
Forecast	Test IC 0.289, out-of-sample R^2 5.5% at 5 s; well calibrated by decile
Taker strategy	815 test trades, gross −0.56 bp, net −10.56 bp per trade
Maker strategy	326 test trades, gross −1.55 bp, net −6.91 bp per trade
Execution overlay	Cost 4.54 bp vs 4.67 bp passive and 5.96 bp immediate (30 s window, t = −8.0)
Engine	4,025,047 events per day replayed in 27 ms; 936 backtests in 17 s; 14 unit tests

How to read this report. Section 2 explains the market-microstructure ideas the project relies on. Sections 3 to 9 follow the pipeline step by step: what each step does, why it is built that way, the command that runs it, and the result it produced. Section 10 lists the limitations and natural extensions.

2. Concepts you need first

2.1 The limit order book and the touch

An exchange keeps a **limit order book**: resting buy orders (bids) and sell orders (asks) at different prices. The highest bid and the lowest ask form the **touch** (level 1). Their difference is the **spread**; the smallest allowed price step is the **tick**. The quantity resting at each touch price is the **L1 size** or depth. The **mid price** is the average of the best bid and ask.

XRPUSDT traded between about US$0.49 and US$0.53 in the sample, and its tick is 0.0001. One tick is therefore about 1.96 bp of price, and the spread was exactly one tick in 99.99% of samples. Assets like this are called **large-tick**: the price is pinned to a one-tick spread, large queues build up at the touch, and the mid moves only when one side's queue is used up. That makes queue sizes very informative about the next price move.

2.2 Three order-flow signals

- **Queue imbalance (QI)** compares the bid and ask queues: QI = $(q_{bid} - q_{ask}) / (q_{bid} + q_{ask})$. It runs from −1 (all size on the ask) to +1 (all on the bid). A thin ask queue is likely to be consumed first, so a high QI predicts an up-tick.
- **Order-flow imbalance (OFI)**, from Cont, Kukanov and Stoikov (2014), adds up the net change in bid-side and ask-side demand at every quote update: $e_n = 1\{P^b_n \geq P^b_{n-1}\} q^b_n - 1\{P^b_n \leq P^b_{n-1}\} q^b_{n-1} - 1\{P^a_n \leq P^a_{n-1}\} q^a_n + 1\{P^a_n \geq P^a_{n-1}\} q^a_{n-1}$. Summed over a window, it measures buying versus selling pressure in the book itself.
- **Trade-flow imbalance (TFI)** uses executed trades: signed aggressor volume divided by total volume over a window. A buy is 'aggressive' when the buyer crossed the spread to hit a resting ask.
- **Recent returns** (the mid change over the last 5 and 30 seconds) test for short-term momentum or reversal.

2.3 Makers, takers, fees and latency

A **taker** crosses the spread with a marketable order and is filled at once, paying the spread and the taker fee (5.0 bp on Binance VIP 0). A **maker** rests a limit order in the book and pays the lower maker fee (2.0 bp), but must wait in the queue and may never be filled. **Latency** is the delay between deciding and the order reaching the exchange; during that time the book can move. A taker round trip on XRP therefore costs one tick of spread plus 10 bp of fees, about 12 bp in total, before any signal is applied.

2.4 Adverse selection and queue position

A resting bid is filled when someone sells into it. That tends to happen exactly when sellers are aggressive and the price is about to fall, so passive fills arrive at the worst moments: this is **adverse selection**. How bad it is depends on **queue position**. An order at the back of a 100,000-contract queue is filled only when the whole level is nearly gone, which is usually just before the price ticks against it.

2.5 Measuring prediction

- **Spearman information coefficient (IC)**: the rank correlation between a feature and the forward mid move. It is computed separately for each day, and days are then averaged within each split.
- **Out-of-sample R²** against a zero forecast: $R^2_{OOS} = 1 - \Sigma(y - \hat{y})^2 / \Sigma y^2$. It is positive only if the forecast beats predicting no move at all.
- **Chronological split**: train on 2024-02-01 to 2024-02-05, validate on 2024-02-06 to 2024-02-07, test on 2024-02-08 to 2024-02-10. Every choice is made before the test days are touched.

2.6 Measuring execution cost

For an order of side s (+1 buy, −1 sell) arriving when the mid is m_0 and filled at price p, the **implementation shortfall** in basis points is: cost = $s \cdot (p - m_0) / m_0 \cdot 10^4$ + fee. The first term is **slippage**. It is about half a tick for a market order, and can be negative for a passive fill.

3. Project layout and setup

The project is a small Python package, `ofalpha`, with one module per pipeline step. Each step reads the previous step's files from disk, so any step can be re-run on its own.



Figure 1. The six-step pipeline. The matching engine (step 4) is the core piece of engineering; everything after it runs on top of it.

File	Role
ofalpha/config.py	Symbol, dates, train/validation/test split, fees, clip size
ofalpha/data.py	Downloads quotes and trades, verifies SHA-256 checksums, writes Parquet
ofalpha/features.py	Point-in-time features and forward targets on a 100 ms clock
ofalpha/research.py	IC analysis, decile tables, OLS forecast, out-of-sample evaluation
ofalpha/engine.py	Numba matching engine: taker, maker and execution-overlay simulators
ofalpha/sweep.py	Parameter sweep, validation selection, single test run
ofalpha/execution.py	Execution-cost study with placebo, latency and block standard errors
ofalpha/report.py	Figures and results/RESULTS.md
tests/test_engine.py	14 hand-built order-book scenarios with known fills
run_all.py	Runs the whole pipeline in order

Setup and full run. Create a virtual environment, install the pinned packages, run the tests, then run the pipeline. The download is about 550 MB; compute takes about seven minutes on a desktop CPU.

```
python -m venv .venv
.venv\Scripts\activate            # Windows (source .venv/bin/activate elsewhere)
pip install -r requirements.txt
python -m pytest -q tests          # 14 matching-engine tests
python run_all.py                  # download -> features -> research -> sweep -> execution -> report
```

Configuration (`ofalpha/config.py`):

Setting	Value	Meaning
SYMBOL	XRPUSDT	Binance USD-M perpetual
TICK	0.0001	price step
DATES	2024-02-01 … 2024-02-10	10 days
TRAIN / VALID / TEST	5 / 2 / 3 days	chronological
GRID_MS	100	sampling and decision clock
WINDOWS_S	[1, 5, 30]	look-back windows for flow features
HORIZONS_S	[1, 2, 5, 10, 30, 60]	forward horizons for targets
MAKER / TAKER_FEE_BP	2.0 / 5.0	Binance VIP 0, Feb 2024
ORDER_QTY	1,000	contracts per order, about US$500

4. Step 1: Getting the data

Binance publishes daily archives of every best-bid/ask change (`bookTicker`) and every aggregated trade (`aggTrades`) at `data.binance.vision`. The bookTicker archive for USD-M futures ends in March 2024, which is why the sample is February 2024.

What the step does. For each day it downloads both zip files, checks them against the published SHA-256 checksum, converts prices to **integer ticks** (so price comparisons in the engine are exact), drops repeated identical quotes, and stores compact Parquet files. Raw zips are processed in memory and never written to disk.

The one non-obvious field is `is_buyer_maker`: when it is true, the buyer was resting in the book, so the *seller* was the aggressor.

```
# is_buyer_maker=True means the seller was the aggressor (hit the bid)
side = np.where(df.is_buyer_maker.astype(bool).to_numpy(), -1, 1).astype(np.int8)
```

```
python -m ofalpha.data            # ~550 MB, prints rows per file
```

Day	Split	Quote updates	Trades	Traded (US$M)	Mean L1 depth	1-tick spread
2024-02-01	train	3,346,146	138,488	429	80,183	99.99%
2024-02-02	train	2,807,855	105,988	329	95,189	100.00%
2024-02-03	train	2,832,485	137,823	445	84,557	100.00%
2024-02-04	train	2,687,357	94,441	280	91,289	99.99%
2024-02-05	train	3,161,443	111,508	366	95,224	100.00%
2024-02-06	valid	3,059,882	102,065	322	88,628	99.99%
2024-02-07	valid	3,204,900	108,700	374	94,274	100.00%
2024-02-08	test	3,147,365	108,226	409	93,043	100.00%
2024-02-09	test	3,399,085	120,746	414	95,139	99.99%
2024-02-10	test	2,814,743	87,209	221	109,266	100.00%
Total		30,461,261	1,115,194	3,590	92,679	99.99%

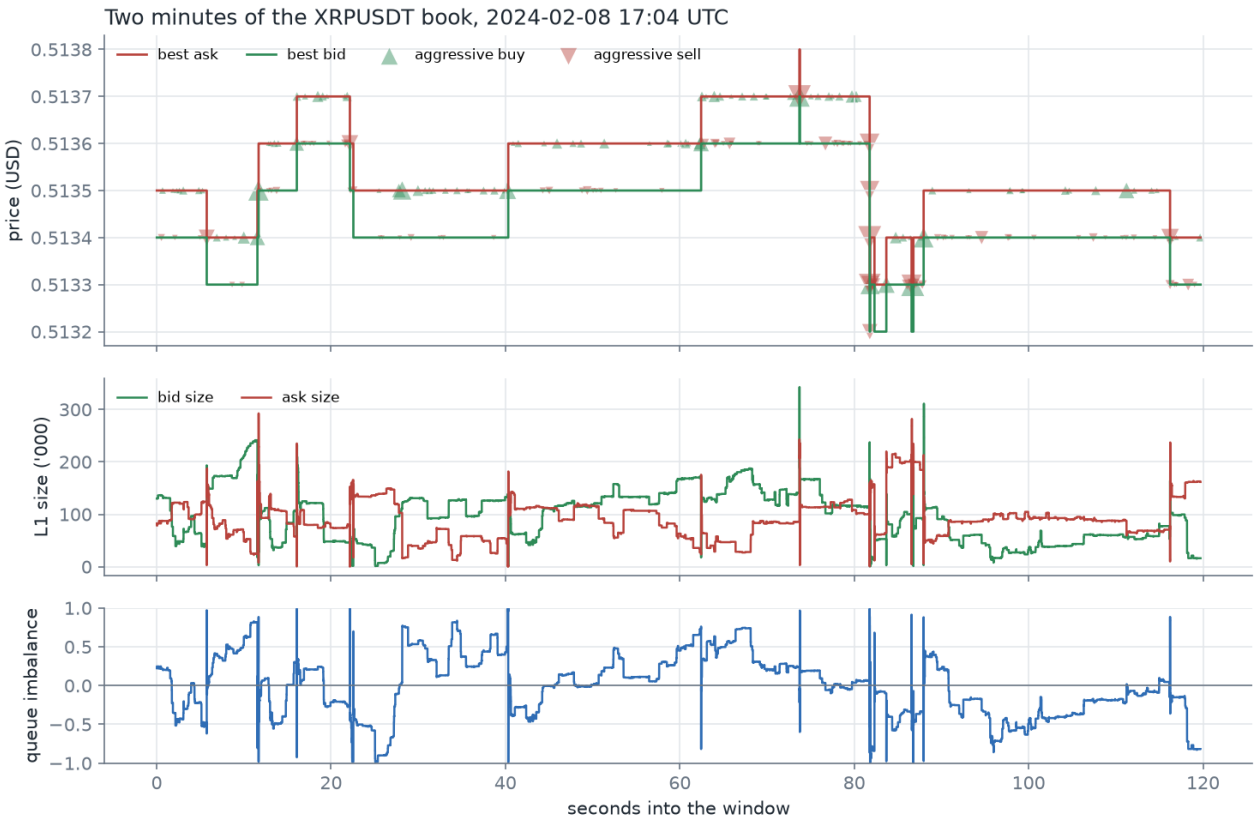


Figure 2. Two minutes of raw data (5,383 quote updates, 195 trades, 33 mid moves). The spread sits at one tick; the mid moves when one side's L1 queue runs out, and queue imbalance (bottom) often leans the right way beforehand.

5. Step 2: Point-in-time features

What the step does. It puts every day on a fixed 100 ms clock (8,631,000 samples in total; the first 30 s and the last 60 s of each day are dropped so every window and horizon is complete). At each clock time g, it takes the **last** quote and trade stamped at or before g. This 'as-of' join is what makes the features point-in-time: no feature can see the future.

```
def asof(times):                                    # last quote with t <= time
    return np.searchsorted(qt, times, side="right") - 1
```

OFI is computed once per quote update with vectorised NumPy, cumulated, and differenced over each window. It is then divided by current L1 depth so that days with deep and shallow books are comparable.

```
def ofi_increments(bid, bq, ask, aq) -> np.ndarray:
    """Per-update order-flow imbalance e_n at the best quotes."""
    e = np.zeros(len(bid))
    b0, b1, a0, a1 = bid[:-1], bid[1:], ask[:-1], ask[1:]
    e[1:] = ((b1 >= b0) * bq[1:] - (b1 <= b0) * bq[:-1]
             - (a1 <= a0) * aq[1:] + (a1 >= a0) * aq[:-1])
    return e
```

Targets are forward mid changes in ticks, read with the same as-of lookup at g + h:

```
for h in C.HORIZONS_S:
    out[f"fwd_{h}"] = (mid2[asof(g + h * 1000)] - mid2[iq]) / 2
```

Column	Definition
qi	(bid size − ask size) / (bid size + ask size) at the touch
ofi_1, ofi_5, ofi_30	Order-flow imbalance over 1, 5 and 30 s, divided by L1 depth
tfi_1, tfi_5, tfi_30	Signed aggressor volume / total volume over 1, 5 and 30 s
ret_5, ret_30	Mid change over the last 5 and 30 s, in ticks
fwd_1 … fwd_60	Target: mid change over the next 1, 2, 5, 10, 30, 60 s, in ticks

```
python -m ofalpha.features        # ~20 s; prints grid points, 1-tick share, mean depth per day
```

6. Step 3: Signal research

6.1 Which features predict, and how far ahead

For each day, feature and horizon, the step computes the Spearman IC on all ~860,000 samples of that day. Averaging per split shows whether predictive power is stable or a fluke of one period.

Feature	Train IC	Validation IC	Test IC
qi	0.273	0.283	0.284
ofi_1	0.143	0.130	0.139
ofi_5	0.136	0.129	0.143
ofi_30	0.076	0.083	0.054
tfi_5	0.058	0.043	0.053
tfi_1	0.044	0.033	0.035
ret_5	0.037	0.019	0.026
tfi_30	0.037	0.042	0.017
ret_30	0.035	0.035	0.005

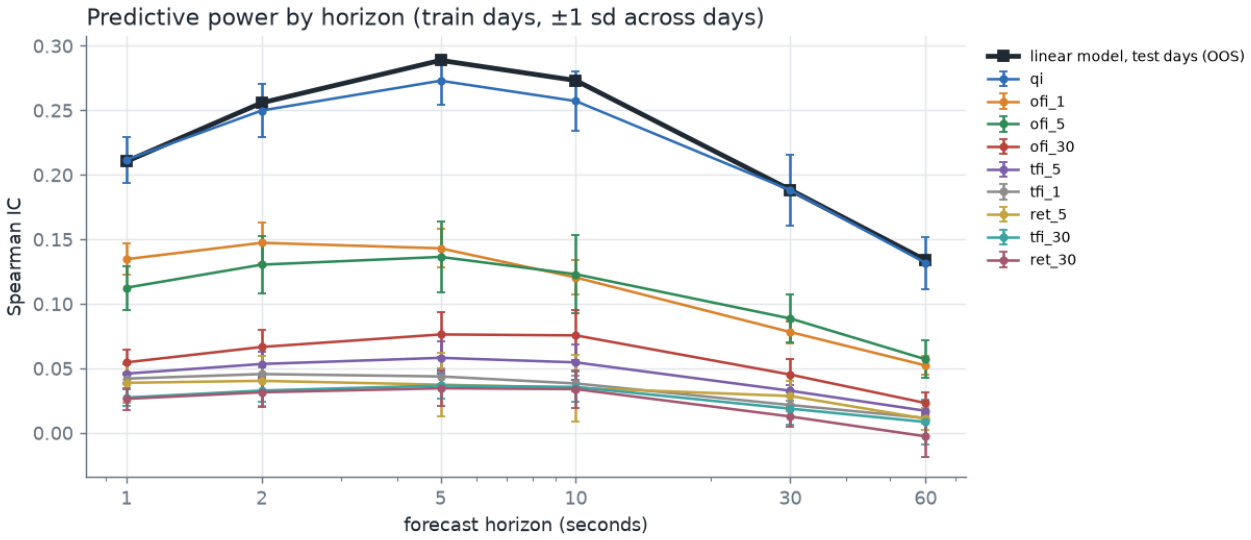


Figure 3. IC by forecast horizon (train days, error bars = one standard deviation across days). Queue imbalance dominates and peaks at 5 seconds; the combined model on test days (black) tracks it closely.

Reading the result
Queue imbalance alone carries most of the information, with an IC near 0.28 at 5 s that is almost identical in all three periods. Order-flow imbalance adds a little; trade flow and past returns are weak. Predictive power peaks at 5 to 10 seconds and halves by 60 seconds: this is a seconds-scale signal.

6.2 How big the predicted moves are

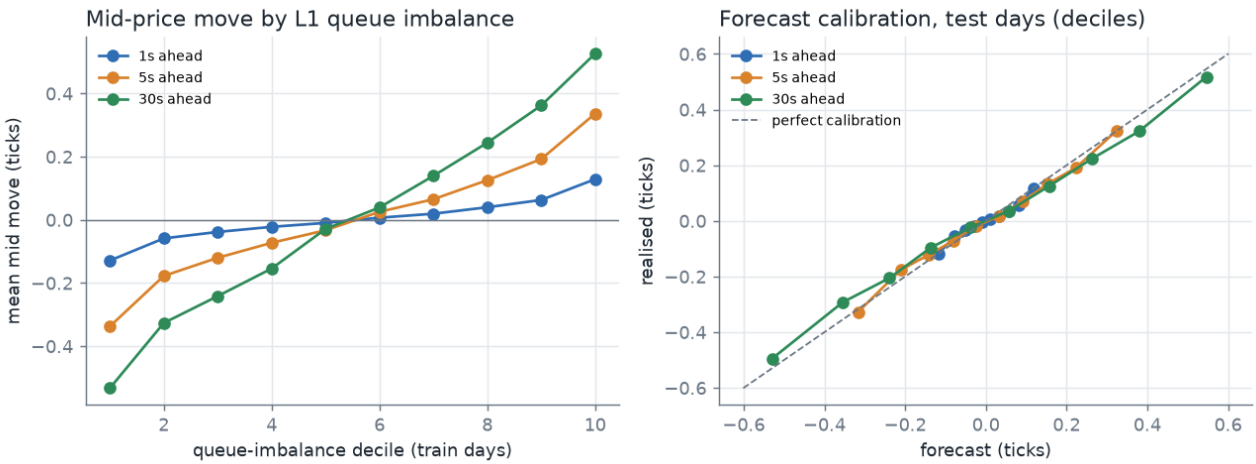


Figure 4. Left: mean forward mid move by queue-imbalance decile (train days). Right: forecast versus realised move by forecast decile on test days; points close to the diagonal mean the forecast's size, not just its sign, is right.

The relationship is monotonic, but the moves are small. In the top QI decile the mid rises on average 0.34 ticks over 5 s and 0.53 ticks over 30 s; in the bottom decile it falls 0.34 and 0.53 ticks. Even the most extreme decile predicts less than one tick, and that is the central fact for everything that follows.

6.3 A combined linear forecast

All nine features are standardised with train-day means and standard deviations, then one OLS regression per horizon is fitted on the train days only. The fitted coefficients are frozen in `results/model.json` and applied unchanged to validation and test days.

```
def fit(days) -> dict:
    df = pd.concat([load(d) for d in days])
    X = df[FEATURES].to_numpy(np.float64)
    mu, sd = X.mean(0), X.std(0)
    Z = np.column_stack([np.ones(len(X)), (X - mu) / sd])
    model = {"features": FEATURES, "mean": mu.tolist(), "std": sd.tolist(), "beta": {}}
    for tgt in TARGETS:
        beta, *_ = np.linalg.lstsq(Z, df[tgt].to_numpy(np.float64), rcond=None)
        model["beta"][tgt] = beta.tolist()
    return model
```

Horizon (s)	R² train	R² valid	R² test	IC train	IC valid	IC test
1	3.22%	1.80%	2.25%	0.216	0.206	0.210
2	4.10%	2.77%	3.76%	0.254	0.250	0.256
5	4.36%	3.87%	5.48%	0.277	0.284	0.289
10	3.63%	3.84%	5.21%	0.259	0.275	0.273
30	1.98%	1.91%	2.58%	0.190	0.192	0.188
60	1.08%	0.79%	1.35%	0.135	0.119	0.134

```
python -m ofalpha.research       # ~80 s; IC table, decile table, model.json, out-of-sample metrics
```

7. Step 4: The matching engine

Vectorised backtests (multiply a signal by next-period returns) ignore exactly the things that decide whether a seconds-scale signal makes money: the spread, queue position, latency and fees. The engine replays the recorded tape event by event instead, and simulates how the strategy's own orders would have been filled. It is written in Numba, so the inner loop compiles to machine code.

7.1 Event loop

- Six event sources are merged in time order: market-order arrivals, cancel arrivals, limit-order activations, trade prints, quote updates and decision ticks on the 100 ms clock.
- Ties at the same millisecond are resolved in that order. Trades come before quotes so that a trade and the quote update it caused are not double-counted against our queue position.
- Every action the strategy takes (send, cancel, market exit) becomes a scheduled event at decision time + latency.

7.2 Fill rules

- **Market orders** fill at the touch at arrival. Any size beyond the displayed L1 quantity pays one extra tick.
- **Limit orders** join the back of the queue at their price; their queue position is tracked as shown in Figure 5.
- A limit order that would cross when it arrives (the book moved through it) is filled as a taker.



Figure 5. The 'risk-averse' queue model. It is deliberately conservative: it never assumes that cancellations happen in front of our order.

The queue logic inside the event loop. A trade print on the other side either trades through our price or eats into the queue ahead of us:

```
elif ev == 3:                                # trade print
    px, qv, sd = tpx[j], tqty[j], tside[j]
    j += 1
    if o_state == 2 and sd == -o_side:       # aggressor on the other side
        if (o_side > 0 and px < o_px) or (o_side < 0 and px > o_px):
            fill, fill_px = True, float(o_px)
        elif px == o_px:
            o_queue -= qv
            if o_queue + qty <= 0.0:
                fill, fill_px = True, float(o_px)
```

A quote update applies the risk-averse rule, and fills us if the opposite quote reaches our price:

```
elif ev == 4:                                # quote update
    b, a, bqq, aqq = bid[i], ask[i], bq[i], aq[i]
    i += 1
    if o_state == 2:
        if o_side > 0:
            if a <= o_px:
                fill, fill_px = True, float(o_px)
            elif o_px == b:
                o_queue = min(o_queue, bqq)
            elif o_px > b:
                o_queue = min(o_queue, 0.0)
        else:
            if b >= o_px:
                fill, fill_px = True, float(o_px)
            elif o_px == a:
                o_queue = min(o_queue, aqq)
            elif o_px < a:
                o_queue = min(o_queue, 0.0)
```

7.3 Unit tests

Test	What it proves
taker_round_trip_pays_the_spread	Taker buys at the ask, sells at the bid; 5 non-overlapping trades
taker_slippage_beyond_displayed_size	Size beyond L1 fills one tick worse (101.5 in, 99.5 out)
maker_fills_only_after_queue_ahead_trades	Fill only after 500 ahead + 100 ours trade; exit at target
maker_no_fill_while_queue_ahead_remains	150 traded against 500 ahead: no fill
displayed_size_drop_moves_us_forward	Displayed size falls to 50, so a 150 trade fills us
trade_through_fills	A trade below our bid fills us at our price
timeout_exits_with_market_order	Maximum holding time triggers a market exit at the bid
stop_loss_exits_at_bid	A 3-tick adverse move with a 2-tick stop exits at 97
execution_immediate_buys_at_ask	Immediate policy pays the ask at arrival + latency
execution_passive_fill_through_queue	Passive child order fills once the queue clears
execution_passive_crosses_at_deadline	Unfilled order cancels at deadline + latency, then crosses
execution_signal_crosses_at_once_...	Signal-aware policy crosses at once when the forecast is adverse
execution_signal_crosses_early_...	Forecast turns at 1,000 ms: crosses at 1,010 ms
execution_sell_side_is_symmetric	Sell orders mirror buy orders

```
python -m pytest -q tests
..............                                          [100%]
14 passed
```

Speed
The maker engine replays 4,025,047 events (one day of quotes, trades and decisions) in 27 ms on a single core, about 148 million events per second. A 936-backtest sweep takes 17 s. Splitting one day's 468 backtests over 18 processes gave only 1.1x, because process start-up and data loading cost more than the compiled runs themselves.

8. Step 5: Standalone strategies

Two ways of trading the forecast directly are tested, each with a parameter grid. Every configuration is run on the two validation days. One configuration per style is selected by validation PnL, and only that configuration is run on the three test days.

- **Taker**: when |forecast| exceeds θ ticks, cross the spread, hold for h seconds, cross back. Grid: 6 thresholds × 6 horizons × 3 latencies = 108 configurations.
- **Maker**: join the touch on the side the forecast favours; cancel if the signal fades or the price leaves; after a fill, post a passive exit one spread away; exit by market order on a time stop, a price stop or a signal flip. Grid: 5 × 4 × 3 × 2 × 3 = 360 configurations.

Style	Configuration selected on validation
Taker	θ = 0.75 ticks, horizon 30 s, latency 10 ms
Maker	θ = 0.2 ticks, horizon 2 s, max hold 120 s, stop 4 ticks, latency 10 ms

Test-day results (per round trip, in basis points of the entry price):

Strategy	Trades	Gross bp	Net bp	PnL (US$)	Win rate
Taker (selected)	815	−0.56	−10.56	−449	2.0%
Maker (selected)	326	−1.55	−6.91	−119	0.2%
Maker, random-sign signal	2155	−1.95	−8.26	−923	0.0%

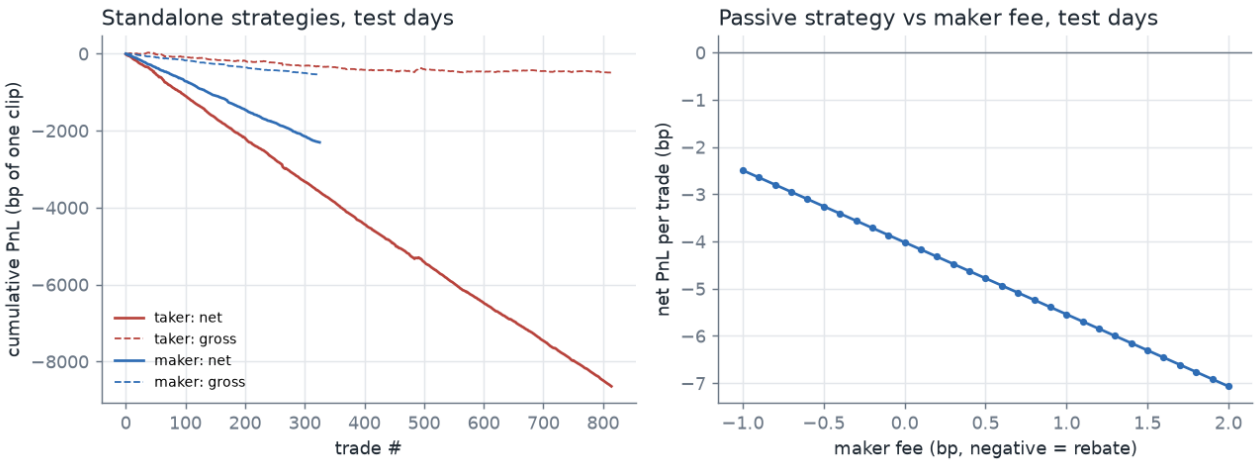


Figure 6. Left: cumulative test-day PnL per trade, gross (dashed) and net (solid). Right: the maker strategy's net PnL per trade as the maker fee varies; it stays negative even with a 1 bp rebate.

Why the maker strategy loses: test-day trades by exit reason.

Exit reason	Trades	Share	Gross bp	Net bp
target	185	57%	0.16	−4.00
signal flip	106	33%	−2.63	−9.69
stop	31	10%	−8.70	−15.89
timeout	3	1%	−4.49	−11.49
end of data	1	0%	−1.94	−8.94

Maker latency sensitivity on test days:

Latency (ms)	Trades	Gross bp	Net bp
1	321	−1.62	−6.89
10	326	−1.55	−6.91
50	344	−1.68	−7.32
100	356	−1.74	−7.56
200	374	−1.75	−7.79
500	436	−1.98	−8.24

Reading the result
The taker loses only 0.56 bp before fees, so the forecast recovers most of the ~2.0 bp spread, but 10 bp of taker fees cannot be overcome. The maker strategy is adversely selected. Passive 'target' exits only scratch, because by the time the order fills the price has usually moved against it, while stops and signal flips lose several bp. With a 1 bp maker rebate it still nets −2.49 bp per trade. The random-sign version is worse (−1.95 bp gross), so the signal does reduce adverse selection, but not by enough.

```
python -m ofalpha.sweep --bench   # benchmark, 936 validation runs, selection, test run
```

9. Step 6: The execution overlay

Section 8 shows the forecast cannot pay for its own trades. Most real trading is not optional, though: a portfolio rebalance or a hedge must be executed anyway, and the question becomes *how cheaply*. This step tests whether the forecast lowers the cost of trades that have to happen.

9.1 Design

- A parent order of 1,000 contracts arrives every 10 seconds with a random side (25,875 orders on the test days) and must be completed within a window W of 5, 10, 30 or 60 seconds.
- **Immediate**: market order on arrival.
- **Passive**: rest at the near touch; if still unfilled at the deadline, cancel and cross.
- **Signal-aware**: as passive, but cross at once (or early, while waiting) whenever the forecast says the price is about to move away from us, that is when side × forecast > θ.
- **Placebo**: the signal-aware rule driven by a forecast with a random sign; it should not help.
- The threshold θ and forecast horizon are chosen on validation days for each W; test days are run once.
- Standard errors of paired cost differences use 5-minute blocks, because orders with overlapping windows are not independent.

```
if policy == IMMEDIATE or (policy == SIGNAL and s * pred[g0] > theta):
    cross_t = t0 + lat_ms
    ...
if T > t0 and s * pred[k - 1] > theta and T + lat_ms < cancel_t:
    cancel_t = T + lat_ms     # pull the order and cross early
```

9.2 Results

Window	Policy	Cost bp	Slippage bp	Passive fills	Avg wait (s)
5 s	immediate	5.96	0.96	0%	0.1
5 s	passive	5.59	1.01	14%	4.7
5 s	signal-aware	5.47	0.84	12%	3.3
5 s	placebo (random-sign signal)	5.83	0.98	5%	2.5
10 s	immediate	5.96	0.96	0%	0.1
10 s	passive	5.31	1.03	24%	8.7
10 s	signal-aware	5.16	0.76	20%	5.6
10 s	placebo (random-sign signal)	5.76	0.99	7%	3.9
30 s	immediate	5.96	0.96	0%	0.1
30 s	passive	4.67	1.06	46%	21.4
30 s	signal-aware	4.54	0.72	39%	14.7
30 s	placebo (random-sign signal)	5.27	1.02	25%	12.6
60 s	immediate	5.96	0.96	0%	0.1
60 s	passive	4.14	1.00	62%	34.8
60 s	signal-aware	4.15	0.68	51%	22.2
60 s	placebo (random-sign signal)	4.88	1.00	37%	19.7

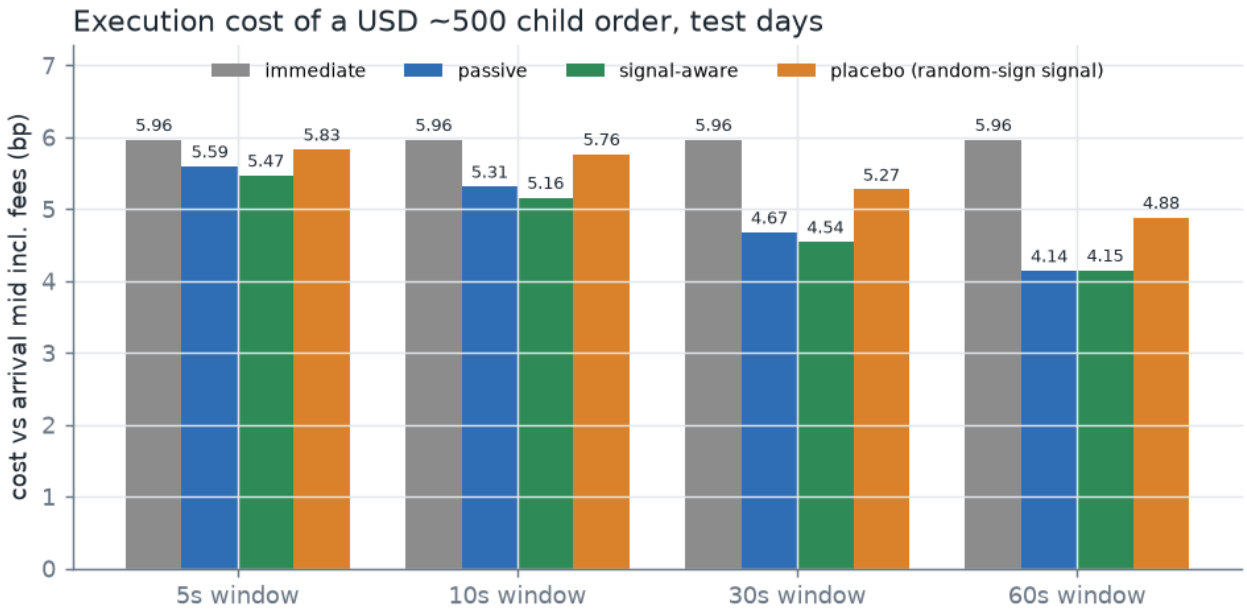


Figure 7. Total cost per order on test days, including fees.

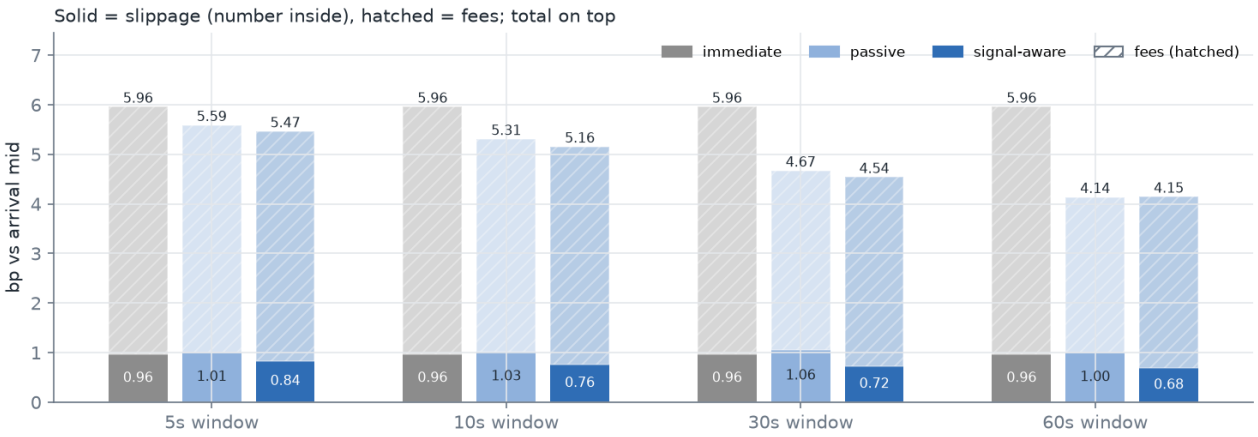


Figure 8. The same costs split into slippage (solid) and fees (hatched). The forecast cuts slippage in every window, but crossing more often pays more taker fees, which absorbs part of the gain.

Paired differences (same orders, negative = cheaper):

Window	Comparison	Difference bp	Std. error	t-stat
5 s	signal-aware minus passive	−0.126	0.007	−18.4
5 s	signal-aware minus immediate	−0.497	0.014	−35.0
5 s	placebo minus passive	0.236	0.012	19.5
10 s	signal-aware minus passive	−0.153	0.010	−15.3
10 s	signal-aware minus immediate	−0.805	0.018	−45.6
10 s	placebo minus passive	0.452	0.017	26.2
30 s	signal-aware minus passive	−0.130	0.016	−8.0
30 s	signal-aware minus immediate	−1.420	0.025	−57.8
30 s	placebo minus passive	0.598	0.022	27.1
60 s	signal-aware minus passive	0.007	0.026	0.3
60 s	signal-aware minus immediate	−1.815	0.029	−62.7
60 s	placebo minus passive	0.742	0.030	24.7
30 s	latency 10 ms: signal-aware minus passive	−0.152	0.016	−9.3
30 s	latency 50 ms: signal-aware minus passive	−0.130	0.016	−8.0
30 s	latency 200 ms: signal-aware minus passive	−0.086	0.016	−5.2
30 s	latency 500 ms: signal-aware minus passive	−0.050	0.016	−3.0

Fee-tier sensitivity: the same simulated fills re-priced under other Binance fee tiers (policy parameters not re-selected):

Window	Fee tier	Immediate	Passive	Signal-aware	Signal − passive
5 s	no fees (pure slippage)	0.96	1.01	0.84	−0.170
5 s	VIP 0	5.96	5.59	5.47	−0.126
5 s	VIP 4	3.96	3.73	3.59	−0.141
5 s	VIP 9	2.66	2.47	2.33	−0.145
10 s	no fees (pure slippage)	0.96	1.03	0.76	−0.267
10 s	VIP 0	5.96	5.31	5.16	−0.153
10 s	VIP 4	3.96	3.55	3.36	−0.191
10 s	VIP 9	2.66	2.32	2.12	−0.203
30 s	no fees (pure slippage)	0.96	1.06	0.72	−0.336
30 s	VIP 0	5.96	4.67	4.54	−0.130
30 s	VIP 4	3.96	3.13	2.94	−0.199
30 s	VIP 9	2.66	1.97	1.75	−0.219
60 s	no fees (pure slippage)	0.96	1.00	0.68	−0.314
60 s	VIP 0	5.96	4.14	4.15	0.007
60 s	VIP 4	3.96	2.76	2.66	−0.100
60 s	VIP 9	2.66	1.65	1.51	−0.132

Reading the result
In the 30-second window the forecast lowers slippage from 1.06 to 0.72 bp (32%) and total cost by 0.130 bp (t = −8.0). The placebo raises cost by 0.60 bp, so direction matters, not only timing. The gain fades with latency, and it grows as fees fall, because cheaper crossing makes acting on the forecast cheaper. In the 60-second window, waiting alone already captures most of the benefit.

```
python -m ofalpha.execution       # ~4 min; selection on validation, test once, placebo, latency
```

10. Limitations and next steps

10.1 Limitations

- One symbol, ten days and one market regime. The three test days give tens of thousands of orders and trades (enough for execution comparisons) but are too short for a strategy-level Sharpe ratio.
- Level-1 data only: queue position is modelled, not observed, and deeper book levels are invisible. The risk-averse queue rule is a deliberately conservative assumption.
- Our own orders are assumed not to move the market. That is reasonable for 1,000-contract clips against about 92,679 contracts at the touch, but not for large size.
- Quote and trade streams are aligned at millisecond resolution; trades are processed first on ties.
- Fee tiers other than VIP 0 are approximate.

10.2 Natural extensions

- **More data**: more symbols (BTC, ETH, SOL) and months, using the same pipeline, to test whether the results generalise.
- **Lead-lag**: use BTC order flow to forecast XRP, a cross-asset signal that may reach further ahead.
- **Queue-position strategies**: join a new price level as soon as it forms, while the queue is short, instead of joining at the back.
- **Longer horizons**: forecast minutes ahead, where predicted moves can exceed one tick.
- **Overfitting audit**: deflated Sharpe ratio and probability of backtest overfitting across the 936-run sweep.

Appendix A. Console output of the sweep

Captured from `results/sweep_log.txt` after a full run:

```
benchmark: 468 runs, serial 10.3s, 18 workers 9.5s, speedup 1.1x
maker replay: 4,025,047 events in 27 ms (148M events/s, one core)
validation sweep: 936 runs in 17.3s
selected on validation: {'taker': {'style': 'taker', 'theta': 0.75, 'horizon_s': 30, 'lat_ms': 10}, 'maker':
  {'style': 'maker', 'theta': 0.2, 'horizon_s': 2, 'lat_ms': 10, 'max_hold_s': 120, 'stop_ticks': 4.0}}
                          trades  gross_bp  net_bp  pnl_usd
style variant     lat_ms
maker latency     1          321    -1.618  -6.893 -115.451
                  10         326    -1.554  -6.914 -119.471
                  50         344    -1.681  -7.325 -134.770
                  100        356    -1.739  -7.556 -144.698
                  200        374    -1.755  -7.785 -156.340
                  500        436    -1.983  -8.239 -196.683
      random sign 10        2155    -1.952  -8.257 -923.129
      selected    10         326    -1.554  -6.914 -119.471
taker selected    10         815    -0.559 -10.559 -448.637
```

Appendix B. Glossary

Term	Meaning
Aggressor	The side that crossed the spread to trigger a trade
As-of join	Matching each time to the latest record stamped at or before it
bp (basis point)	One hundredth of a percent, 0.01%
IC	Information coefficient; here the Spearman rank correlation with the forward move
Implementation shortfall	Cost of a trade measured against the mid price when the order arrived
Large-tick asset	Instrument whose spread is pinned at one tick with deep queues
Latency	Delay between a decision and the order reaching the exchange
Maker / taker	Resting liquidity provider / immediate liquidity consumer
OFI	Order-flow imbalance (Cont, Kukanov and Stoikov, 2014)
Placebo	A version of a test with the signal's information removed, used as a control
Queue imbalance	(bid size − ask size) / (bid size + ask size) at the touch
Touch / L1	The best bid and best ask